

Interface - Intermediate

- I2C
 - Concepts : bus, speed, pins, configuration : hw/sw (pullup etc)
 - Limitation?
 - Exercise : LCD display with PF85xx
- SPI
 - Concept
 - Exercise : read / write W25Q31

Daftar Percobaan I²C (Sensor BH1750 & GY-91)

- **Pemindaian Jalur (I2C Scanner):** Menjalankan program pemindai alamat untuk mendeteksi sensor BH1750 dan GY-91 yang dirangkai secara paralel pada satu *bus* I²C (pin SDA dan SCL), dan berhasil melacak alamat heksadesimal keduanya beroperasi bersamaan tanpa saling mengganggu.
- **Resolusi Konflik Alamat:** Melakukan manipulasi *hardware* dengan mengubah logika tegangan pada pin ADDR di modul sensor (menariknya ke jalur HIGH atau LOW). Percobaan ini membuktikan bahwa dua sensor yang bentuknya identik bisa diubah alamat *software*-nya agar tidak bertabrakan di satu *bus*.
- **Uji Kecepatan Optimal (BH1750):** Mengubah parameter kecepatan *clock* I²C dan berhasil mencapai titik operasional tinggi di 88.000 Hz, yang menghasilkan durasi penarikan data sensor pencahayaan dalam waktu 120 mikrodetik.
- **Uji Ekstrem Underflow (GY-91):** Memasukkan parameter kecepatan di luar spesifikasi fisik Arduino Uno, yaitu 10 MHz. Percobaan ini membuktikan terjadinya limitasi ukuran *register* (*integer underflow*), di mana waktu pembacaan GY-91 justru membengkak menjadi 2524 mikrodetik.

Daftar Percobaan SPI (Modul W25Q32)

- **Validasi Perangkat Keras (JEDEC ID):** Mengirimkan instruksi *raw* 0x9F ke cip memori untuk memvalidasi sambungan 4 kabel (CS, MOSI, MISO, SCK). Percobaan sukses mendeteksi identitas pabrikan Winbond (0xEF), tipe memori (0x40), dan kapasitas 4 Megabyte (0x16).
- **Siklus Baca-Tulis Memori Flash:** Mengeksekusi urutan hukum fisika memori *Flash* secara murni tanpa *library* pihak ketiga. Urutan yang dilakukan meliputi pembukaan akses (*Write Enable*), penghapusan sektor blok silikon (*Sector Erase*), penulisan teks "SPI" (*Page Program*), dan pembacaan kembali untuk memverifikasi keberhasilan penyimpanan.
- **Uji Kecepatan Continuous Read (Non-Blocking):** Membuat program *State Machine* 100% *non-blocking* untuk menarik 1000 *byte* data kosong secara beruntun dari alamat awal. Percobaan secara langsung membandingkan waktu tempuh *clock* 1 MHz (9296 mikrodetik) dengan kecepatan maksimal Arduino Uno 8 MHz (2232 mikrodetik).

Kesimpulan Eksperimen I²C

- **Eksperimen *Bus & I2C Scanner*:** Kita telah berhasil merangkai 4 komponen I2C berbeda (**BH1750** x2 dan **GY-91**) dan LCD di atas satu jalur *bus* I²C (SDA & SCL) yang sama. Program *I2C Scanner* juga berhasil mendeteksi alamat heksadesimal kedua perangkat tersebut di dalam jaringan secara akurat tanpa saling bertabrakan.
- **Resolusi Konflik Alamat I²C:** Berdasarkan percobaan, konflik alamat (saat menggunakan dua sensor yang identik) dapat diselesaikan dengan memanipulasi perangkat keras. Kita harus mengubah status **Pin ADDR** pada modul (menyambungkannya ke HIGH atau LOW) untuk mengubah alamat perangkat lunaknya (misalnya pada BH1750 dari 0x23 menjadi 0x5C), sehingga keduanya bisa hidup berdampingan di satu *bus*.
- **Limitasi *Hardware* & Uji Kecepatan:** Pada pengujian sensor BH1750, kita berhasil mengoptimalkan kecepatan hingga **88.000 Hz (88 kHz)** dengan rekor waktu penarikan data yang sangat singkat, yaitu sekitar **120 mikrodetik**.
- **Bug Siluman (*Integer Underflow*):** Saat menguji sensor kelas industri GY-91 dengan menginput kecepatan ekstrem 10 MHz, Arduino tidak *crash*. Namun, limitasi ukuran *register hardware* memicu *underflow*, yang justru menyebabkan kecepatan anjlok parah menjadi sekitar 31 kHz (waktu baca membengkak menjadi ~2524 mikrodetik).

Kesimpulan Eksperimen SPI

- **Kecepatan Brutal & Batas Arduino:** Komunikasi perangkat keras SPI pada Arduino Uno memiliki batas maksimal **8 MHz**. Dengan fitur *Auto-Increment* dan arsitektur *Full-Duplex* (jalur ganda: MOSI & MISO), kita berhasil menyedot 1000 *byte* data secara beruntun dari cip **W25Q32** hanya dalam waktu **~2,2 milidetik** (2232 mikrodetik).
- **Bottleneck pada CPU:** Pada kecepatan maksimal 8 MHz, transmisi data tidak lagi dihambat oleh jalur SPI, melainkan oleh *Software Overhead* (waktu yang dihabiskan oleh CPU Arduino untuk berputar mengeksekusi instruksi kodingan).
- **Hukum Fisika *Flash Memory*:** Memori *Flash* tidak mengizinkan penyimpanan data secara instan. Blok silikon wajib dikosongkan terlebih dahulu menjadi 0xFF (bit 1) melalui perintah *Sector Erase*, sebelum data baru bisa disuntikkan (bit 0) menggunakan perintah *Page Program*.
- **Sistem Pengalamatan:** SPI tidak menggunakan sistem alamat *software*. Proses pemilihan target jauh lebih instan karena murni menggunakan kontrol perangkat keras, yaitu dengan menarik **Pin Fisik CS (*Chip Select*)** ke LOW.